# DocGen_Platform_Technology

# Salesforce Platform Technology Details

## AppExchange Security Review Documentation — v2.1.0 Re-submission

**Package:** Portwood DocGen Managed **Namespace:** `portwoodglobal` **Version:** 2.1.0 **Package Version Id:** `04tVx000000Zw5xIAC` **Released:** Yes (promoted 2026-05-22) **Prior listing version (AppExchange):** v1.56.0 (`04tal000006i1rNAAQ`) — this submission addresses all 30 findings returned by the AppExchange security review against the v1.56 listing. The v2.0/v2.1.0 source tree also rolls forward ~45 versions of feature work since v1.56 (V3 query trees, chart engine, signature v3 with PIN second factor + multi-signer + guided placements, HTML templates, giant-query batching, and more). See `SECURITY_REVIEW_RESPONSE_v2.md` in this folder for the per-finding security map.

**Install URLs:**

- **Production:** https://login.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC
- **Sandbox:** https://test.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC
- **CLI:** `sf package install --package 04tVx000000Zw5xIAC --wait 10 --target-org <your-org>`

> Response to the security review prompt: *"If your solution contains Salesforce Platform technology, such as Lightning Components and Apex, provide details."*

---

DocGen is built **100% on native Salesforce Platform technology**. There are no external services, no callouts, no third-party JavaScript libraries, and no external hosted assets. All processing occurs inside the customer's Salesforce org.

---

# 1. Apex

**42 non-test Apex classes** plus **29 test classes** (1,449 Apex tests, **76% org-wide code coverage**, Code Coverage Met: true at package build time). v2.1.0 adds `DocGenFlsGuard` + `DocGenFlsGuardTest` over the v2.0 baseline of 41/28.

## Controllers (`@AuraEnabled`, entry points from LWC / VF)

| Class | Sharing | Purpose |
|---|---|---|
| `DocGenController` | `with sharing` | Primary generation entry point. Called from `docGenRunner` LWC. |
| `DocGenBulkController` | `with sharing` | Bulk job creation, progress polling, analysis. |
| `DocGenChartImageController` | `with sharing` | Chart rendering pipeline (v1.99 chart engine — 9 styles, pure-Apex PNG). |
| `DocGenTemplateManager` | `with sharing` | Template body retrieval (Word/ HTML CV body fetch). |
| `DocGenSetupController` | `with sharing` | First-run setup wizard + Command Hub metadata. |
| `DocGenSignatureSenderController` | `with sharing` | Admin-initiated signature request creation. |
| `DocGenSignatureController` | `without sharing` | Guest-facing signing page entry point. Token + PIN gated. Wired to `DocGenSignatureGuestSecurity`. |
| `DocGenAuthenticatorController` | `without sharing` | Public document verification by SHA-256 hash (used by `docGenAuthenticator` LWC + `DocGenVerify`). |

## Service / Helper Classes

`DocGenService` (merge engine, ~10K lines), `DocGenDataRetriever` (V1/V2/V3 SOQL), `DocGenChartBucketResolver` (chart aggregate SOQL), `DocGenChartRasterizer` + `DocGenChartFont` + `DocGenChartTagExpander` + `DocGenSvgChartSerializer` (PNG-via-CV chart pipeline), `DocGenHtmlRenderer` (OOXML → HTML for `Blob.toPdf()`), `DocGenPngEncoder` (hand-coded PNG encoder for chart rasterizer), `BarcodeGenerator` (Code-128 / QR), `DocGenSignatureService` (signature stamping + PDF queueable), `DocGenSignatureEmailService` (OWA-branded signing emails), `DocGenApprovalHistory`, `DocGenException`, `DocGenSignatureGuestSecurity` *(shipped in v2.0)*, `DocGenFlsGuard` *(NEW in v2.1.0 — per-field FLS describe-check helper; 243 call sites across 19 controllers)*.

## Asynchronous Apex

- **Batchable:** `DocGenBatch`, `DocGenGiantQueryBatch`

- **Queueable:** `DocGenGiantQueryAssembler`, `DocGenGiantQueryStitchJob`, `DocGenMergeJob`, `DocGenSignatureService.SignaturePdfQueueable`, `TemplateSignaturePdfQueueable`
- **Schedulable:** `DocGenSignatureReminderSchedulable` (hourly auto-reminders for unsigned recipients)
- **Platform Event triggered:** `DocGenSignaturePdfTrigger` fires on `DocGen_Signature_PDF__e` insert → enqueues `SignaturePdfQueueable` to finalize the signed PDF asynchronously (decouples guest-context signing from the merge/email pipeline which runs as Automated Process).

## Flow Invocable Actions (`@InvocableMethod`)

- `DocGenFlowAction` — single-record document generation
- `DocGenBulkFlowAction` — bulk generation against a saved query
- `DocGenGiantQueryFlowAction` — multi-million-row query job
- `DocGenSignatureFlowAction` — create a signature request and return per-signer signing URLs for Flow-driven signature automation

## Apex Triggers

- **`DocGenSignaturePdfTrigger`** — fires on `DocGen_Signature_PDF__e` platform event insert; enqueues `TemplateSignaturePdfQueueable` to finalize the signed PDF asynchronously.

## SOQL / DML Execution Mode — v2.0 / v2.1.0 hybrid pattern

v2.1.0 ships a **three-layer CRUD/FLS enforcement pattern** across every admin endpoint. The v1.56 reviewer's stated finding-resolution language was: *"enforce CRUD checks on the object **AND** FLS checks on the fields before performing any DML operation, **or** alternatively use USER_MODE."* v2.0 implemented the object-level half; v2.1.0 adds the per-field half line-for-line.

- **Layer 1 — Object-level CRUD gate** at every `@AuraEnabled` / `@InvocableMethod` entry point (shipped in v2.0):

  ```
  if (!Schema.sObjectType.DocGen_Template__c.isAccessible()) {
      throw new DocGenException('Insufficient access to DocGen templates.');
  }
  ```

  This is the documented enforcement signal the AppExchange `sfge:ApexFlsViolation` rule pattern-matches on; it gates the entire method body on the user's DocGen permission-set assignment.

- **Layer 2 — Per-field FLS describe-check via `DocGenFlsGuard`** (NEW in v2.1.0). Called immediately before every DML and `WITH SYSTEM_MODE` SOQL site. Performs an explicit `Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()` per field in the operation's allowlist; throws `DocGenException` if any field is denied. **243 call sites across 19 controllers** (70 DML `assertCreateable/assertUpdateable` sites across 9 controllers + 173 SOQL `assertAccessible` sites across 18 classes). Example:

```
DocGenFlsGuard.assertCreateable(job, new Set<String>{
    'Template__c', 'Status__c', 'Query_Condition__c'
});
/* code-analyzer-suppress ApexFlsViolation */
Database.insert(job, AccessLevel.SYSTEM_MODE);
```

- **Layer 3 — `WITH SYSTEM_MODE` SOQL + `AccessLevel.SYSTEM_MODE` DML** for the actual operation, with `/* code-analyzer-suppress ApexFlsViolation */` and inline justification. USER_MODE strict-FLS strips package-namespaced custom fields like `Query_Config__c`, `Header_Html__c`, `Page_Margins__c`, `Content_Version_Id__c` when subscriber admin profiles haven't been granted FLS individually on each new render-config field added across releases (the package-build test orgs reproduce this with `No such column 'Query_Config__c'` errors).

- **Standard objects** (`ContentVersion`, `ContentDocumentLink`, `User`, `OrgWideEmailAddress`, etc.) continue to use **`WITH USER_MODE`** — no namespaced FLS issue there, and no DocGenFlsGuard either (platform-enforced FLS).

- **Guest signing paths** retain `SYSTEM_MODE` (guests structurally have no DocGen CRUD by design, and granting them CRUD would create the cross-tenant data-exposure vulnerability the reviewer is concerned about). The **`DocGenSignatureGuestSecurity`** helper class (shipped in v2.0) centralizes the explicit Schema-CRUD describe checks + token-bound capability validation + per-operation field allowlists at every guest entry point; v2.1.0 layers DocGenFlsGuard per-field guards on top at each guest DML/SOQL. See `SECURITY_REVIEW_RESPONSE_v2.md` § "CRUD/FLS Enforcement — Guest endpoints" for the full per-finding rebuttal.

The prior `// CxSAST: USER_MODE not viable in managed package (namespace resolution breaks unqualified field names); CRUD/FLS enforced by permission sets` rationalizations carried over from v1.56 are **gone** — they were demonstrably wrong (USER_MODE compiles fine in managed packages; the actual issue is FLS-propagation timing in package-build orgs). v2.0/v2.1.0 replaces them with explicit Schema gates AND per-field describe checks that the analyzer accepts.

**Honest scanner disposition**: `sf code-analyzer` (Security + AppExchange selectors) reports **0/0/0** (Critical/High/Moderate) against the v2.1.0 source tree. Checkmarx CxSAST will continue to flag FLS Create / FLS Update / USER_MODE Missing patterns in its next scan because the scanner doesn't trace into the `DocGenFlsGuard` helper class — the rebuttal in `DocGen_False_Positive_Report.md` points at the 243 helper call sites where the explicit per-field describe checks happen.

---

# 2. Lightning Web Components

**18 Lightning Web Components** (no Aura, no Lightning Component Framework):

**Record-page and app components:**

- `docGenRunner` — document generation button on record pages. Includes two pure-JS modules: `docGenZipWriter.js` (dependency-free ZIP writer, CRC-32 inline, store mode) and `docGenPdfMerger.js`. Used for client-side DOCX/XLSX/PPTX assembly.
- `docGenCommandHub` — DocGen app landing page with quick actions, template library, bulk runner.
- `docGenAdmin` — template CRUD and version management. Wired to the chart engine (v1.99) and signature builder (v1.42+).

**Template builders:**

- `docGenSetupWizard`, `docGenAdminGuide`, `docGenQueryBuilder`, `docGenColumnBuilder`, `docGenTreeBuilder`, `docGenTreeNode`, `docGenFilterBuilder`, `docGenTitleEditor`

**Bulk + signatures + verification:**

- `docGenBulkRunner`, `docGenSignatureSender`, `docGenSignatureSettings`, `docGenSharing`, `docGenAuthenticator` (drag-and-drop SHA-256 verifier), `docGenUtils`

### Clickjacking remediation (shipped in v2.0, persists in v2.1.0)

All `style="position: absolute|fixed"` inline attributes on exposed LWCs replaced with the SLDS `slds-is-absolute` utility class + named CSS classes (`.dg-suggestion-dropdown`, `.dg-provider-dropdown`, `.dg-merge-suggestions`, `.dg-drop-overlay`, `.dg-grandchild-dropdown`, `.dg-dropdown`). Five bundles touched: `docGenAdmin`, `docGenAuthenticator`, `docGenBulkRunner`, `docGenQueryBuilder`, plus `docGenColumnBuilder` (consumed by `docGenAdmin` — same threat surface). New `docGenAuthenticator.css` created to host the supporting rules.

**LWS compliance:** All components are Lightning Web Security compatible. No `eval`, no `Function` constructor, no dynamic imports, no access to global window APIs beyond standard typed arrays and `crypto.subtle.digest` (SHA-256 in the verifier). User-supplied strings are rendered via `{expression}` interpolation (auto-escaped) — no `innerHTML` on user data.

---

# 3. Visualforce Pages

**4 Visualforce pages**, all internal to the package (no Sites or public hosting assumed by the package itself — the customer optionally hosts the signing page on their own Salesforce Site).

| Page | Purpose | Access |
|---|---|---|
| `DocGenGuide.page` | In-app admin guide | Admin + User permission sets |
| `DocGenSign.page` / `DocGenSignature.page` | Public signing page (served via customer's Salesforce Site). Same | |

| Page | Purpose | Access |
|---|---|---|
| | `DocGenSignatureGuestSecurity` Schema-gates as the LWC path. | Guest via `DocGen_Guest_Signature` permission set |
| `DocGenVerify.page` | Document integrity verification (SHA-256 hash recomputed locally in browser — file never uploaded). Returns ALL signers for a multi-signer doc (was LIMIT 1 pre-v2.0). | Guest + Admin + User |

All pages use standard Visualforce auto-escaping on all merge fields. URL parameters are validated before reflection.

---

# 4. Custom Objects

**11 custom objects** (9 sObjects + 2 platform events):

| Object | Purpose |
|---|---|
| `DocGen_Template__c` | Logical template definition |
| `DocGen_Template_Version__c` | Versioned OOXML artifact (master-detail to Template) |
| `DocGen_Saved_Query__c` | Reusable V1/V2/V3 query config |
| `DocGen_Job__c` | Bulk generation job tracking |
| `DocGen_Settings__c` | Hierarchy custom setting for org-wide config (branding, OWA, etc.) |
| `DocGen_Signature_Request__c` | Parent of a signature request |
| `DocGen_Signer__c` | One per signer (token, PIN hash, status) |
| `DocGen_Signature_Placement__c` | Per-tag signature placement (guided signing — v3 tag types) |
| `DocGen_Signature_Audit__c` | Immutable audit record with field history tracking |
| `DocGen_Signature_PDF__e` | Platform event for async signed-PDF finalization |
| `DocGen_Guest_Render__e` | Platform event for async guest-context document rendering (signature preview pipeline) |

All relationships between DocGen objects use master-detail (`ControlledByParent` sharing) where appropriate to enforce parent-record-based access.

---

# 5. Permission Sets

**4 permission sets** define the complete CRUD/FLS/tab/page/class access model:

| Permission Set | Target | Scope |
|---|---|---|
| `DocGen_Admin` | Admins | Full CRUD on all DocGen objects; access to all Apex classes, tabs, pages, and the Settings custom setting. Required for template management, bulk job creation, signature sending, and chart rendering. |
| `DocGen_User` | End users | Generate documents from record pages, view own jobs, read templates. Explicitly **denied** on `DocGen_Signer__c.PIN_Hash__c`, `Secure_Token__c`, `PIN_Attempts__c`, `PIN_Expires_At__c`, and `DocGen_Signature_Request__c.Secure_Token__c`. |
| `DocGen_Guest_Runner` | Site guest user (document render) | Read templates + render documents in guest-Experience-Cloud contexts. No write access to template metadata. Used by Experience Cloud guest portals offering self-service document generation. |
| `DocGen_Guest_Signature` | Site guest user (signing) | Minimal scope: read on signature objects exclusively through token-gated entry points in `DocGenSignatureController` (now also gated by `DocGenSignatureGuestSecurity`). No access to templates, jobs, or unrelated record data. |

---

# 6. Custom Application and Tabs

- **1 Custom App** — `DocGen` (Lightning App with Command Hub, Template Manager, Bulk Gen, Setup, and Job History tabs)
- **11 Custom Tabs** — Command Hub, Template Manager, Bulk Gen, Setup, Admin Guide, plus object tabs for Job, Template, Template Version, Signature Request, Signer, and Signature Audit

---

# 7. Flows (Sample)

**2 sample Flows** shipped with the package as admin-editable starting points:

- `DocGen_Generate_Account_Summary` — demonstrates `DocGenFlowAction` usage
- `DocGen_Welcome_Pack_New_Contact` — record-triggered Flow on Contact insert

---

# 8. External Technologies Used

**None.** Specifically:

- **External callouts:** None. Zero Remote Site Settings, zero Named Credentials, zero `Http.send()` invocations. Confirmed by Salesforce Code Analyzer.

- **Third-party JS libraries:** None. `docGenZipWriter.js` is implemented from scratch in the package precisely so no external ZIP library is pulled in. The chart engine's PNG encoder (`DocGenPngEncoder.cls`) is also hand-implemented from scratch — no graphics library dependency.
- **External fonts / images / CSS:** None. No CDN fetches, no `@font-face` from external URLs. The chart engine's Arial-equivalent font glyphs are baked into Apex constants (`DocGenChartFont.cls`).
- **Session ID usage:** None. The package never calls `UserInfo.getSessionId()`.
- **Cryptographic dependencies:** Only Salesforce's built-in `Crypto` class (`generateAesKey`, `generateDigest`, `getRandomInteger`). No hardcoded secrets, keys, or tokens anywhere in the source.

---

# 9. Test Coverage

- **1,449 Apex tests**, 100% pass rate
- **Package build coverage:** 76% (Code Coverage Met: true)
- **Release gate coverage:** 11 end-to-end anonymous Apex scripts (`scripts/e2e-*.apex`) covering permissions, template CRUD, PDF generation, DOCX generation, bulk, signatures, four merge-tag syntax suites, and cleanup. **All 11 scripts pass with 0 failures** on the v2.1.0 release validation run.

---

# 10. Salesforce Code Analyzer Results

Run with `sf code-analyzer run --rule-selector Security --rule-selector AppExchange` against the full `force-app/` tree:

- **0 Critical**
- **0 High**
- **0 Moderate**
- **0 Low**
- **0 Info**

Achieved by two mechanisms:

1. **562 findings genuinely closed (real runtime enforcement)** — FLS Create 118 + FLS Update 104 + USER_MODE Missing 340 — via the v2.1.0 `DocGenFlsGuard` helper, called at 243 sites across 19 controllers.
2. **38 findings disabled at rule level in `code-analyzer.yml`** (full structural justification in the YAML):
   - 9× `pmd:AvoidLwcBubblesComposedTrue` on `docGenTreeNode.js`. `composed: true` is structurally required for events to cross nested shadow DOMs in the recursive tree component (template hierarchy editor). Removing it would break the component.
   - 29× `pmd:ProtectSensitiveData` on field metadata. The rule heuristically flags ANY field name containing `Signer`, `Token`, `Hash`, etc. as "potential auth token" — but in our case these are LITERALLY signature/audit fields where exposing those names IS the point (the audit-trail UI displays signer names). The actually-sensitive token/hash fields (`Secure_Token__c`,

`PIN_Hash__c`) are protected by `DocGen_User` permission-set FLS denial (see § 5 above) and stored as SHA-256 hashes, never plaintext.

See `DocGen_Code_Analyzer_Report.md` (this folder) for the full finding-by-finding disposition and the `code-analyzer.yml` rule disables.

---

# 11. v2.0 → v2.1.0 Security Re-submission Summary

This package version addresses every finding from the prior security review:

## v2.0 (security pass v1 — already shipped)

- **4 clickjacking findings** → SLDS `slds-is-absolute` class swap across 5 LWC bundles (see § 2 above).
- **9 CRUD/FLS findings on admin DocGenController DML methods** → hybrid Schema-CRUD-gate + SYSTEM_MODE pattern (see § 1 SOQL/DML Execution Mode, Layer 1).
- **6 CRUD/FLS findings on DocGenSignatureSenderController** → same hybrid pattern; ContentDistribution inserts now go through `Database.insert(dist, AccessLevel.USER_MODE)` (standard object).
- **10 CRUD/FLS findings on DocGenSignatureController (guest)** → `DocGenSignatureGuestSecurity` helper centralizes the per-operation field allowlist + Schema describe checks + token-bound capability validation. SYSTEM_MODE retained on guest paths because guests structurally cannot have DocGen CRUD.
- **1 CRUD/FLS finding on DocGenBulkController** → hybrid pattern applied.

Plus proactive hardening of code the reviewer did NOT flag in the v1.56 listing review but was the same pattern: `DocGenChartImageController`, `DocGenSetupController`, `DocGenTemplateManager`, both flow actions. All use the same Schema-gate + SYSTEM_MODE hybrid.

Plus one in-flight bug fix during the v2.0 pass: `DocGenAuthenticatorController.verifyDocument` now returns `List<VerificationResult>` (not LIMIT 1) so a multi-signer document drops onto the verifier and shows the full audit trail for every signer.

## v2.1.0 (security pass v2 — per-field FLS guard layer)

- **562 Checkmarx findings closed** (FLS Create 118 + FLS Update 104 + USER_MODE Missing 340) via the new `DocGenFlsGuard` helper. 243 call sites across 19 controllers (70 DML guards in 9 controllers + 173 SOQL guards in 18 classes). Each guard performs `Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()` per field in the operation's allowlist (Layer 2 in § 1 SOQL/DML Execution Mode). This implements the AND half of the v1.56 reviewer's stated finding-resolution language.

- **38 documented false positives suppressed at rule level** in `code-analyzer.yml`:
  - `pmd:ProtectSensitiveData` (29) — field-name pattern matches on signature/audit/branding fields; structural protection via permset FLS denial + SHA-256 hashing at rest for the actually-sensitive ones.
  - `pmd:AvoidLwcBubblesComposedTrue` (9) — `composed: true` structurally required for the recursive `docGenTreeNode` LWC to bubble events to the parent tree builder across shadow DOM boundaries.
- **Honest disclosure**: `sf code-analyzer` (Security + AppExchange) reports **0/0/0** against the v2.1.0 source tree. Checkmarx CxSAST will continue to flag the FLS Create/Update/USER_MODE Missing patterns because the scanner doesn't trace into the `DocGenFlsGuard` helper class; the rebuttal in `DocGen_False_Positive_Report.md` points at the 243 helper call sites.

---

# 12. Related Documentation

- `DocGen_Security_Architecture.md` (this folder) — security-focused architecture, threat model, sharing model, controls matrix.
- `DocGen_Architecture_and_Usage.md` (this folder) — feature/component inventory and usage walkthroughs.
- `DocGen_False_Positive_Report.md` (this folder) — per-category disposition of the analyzer findings.
- `DocGen_Code_Analyzer_Report.md` (this folder) — Salesforce Code Analyzer run results.
- `SECURITY_REVIEW_RESPONSE_v2.md` (this folder) — per-finding map for this re-submission with commit references.

---

*Portwood Global Solutions — https://portwood.dev*